DOCUMENTATION

1. Planning
   Before any code, we need to build a plan.
   I used gemini for this. I attached the requirement files recieved by email, and explained i am using opencode. I asked him to generate a well thought out prompt to give to opencode.
   https://opncd.ai/share/SYBh3NCy
2. Building

Phase 1: Foundation & DB Connection

When I tried to build the base of the app, the AI made a mistake in the code, so when I tried to connect to local host to test it out, i would get this error :

Fatal error: Uncaught Error: Call to undefined method stdClass::dispatch() in /Users/vladnaimaer/Downloads/minirank/public/index.php:36 Stack trace: #0 {main} thrown in /Users/vladnaimaer/Downloads/minirank/public/index.php on line 36

I pointed that out to the AI, and it fixed the proplem.

I then started the php server using this command :
`php -S localhost:8000 -t public`
Opened this link : `http://localhost:8000`
and i got the message that its working
https://opncd.ai/share/m5y3oF2V

Phase 2: Database Schema Execution

I used opencode to make him generate a script so i can generate a database.
Here's how it went :
https://opncd.ai/share/xvsFoNhf
I then built the database using the script created by open code using this command :
sqlite3 data/minirank.sqlite < db/schema.sql

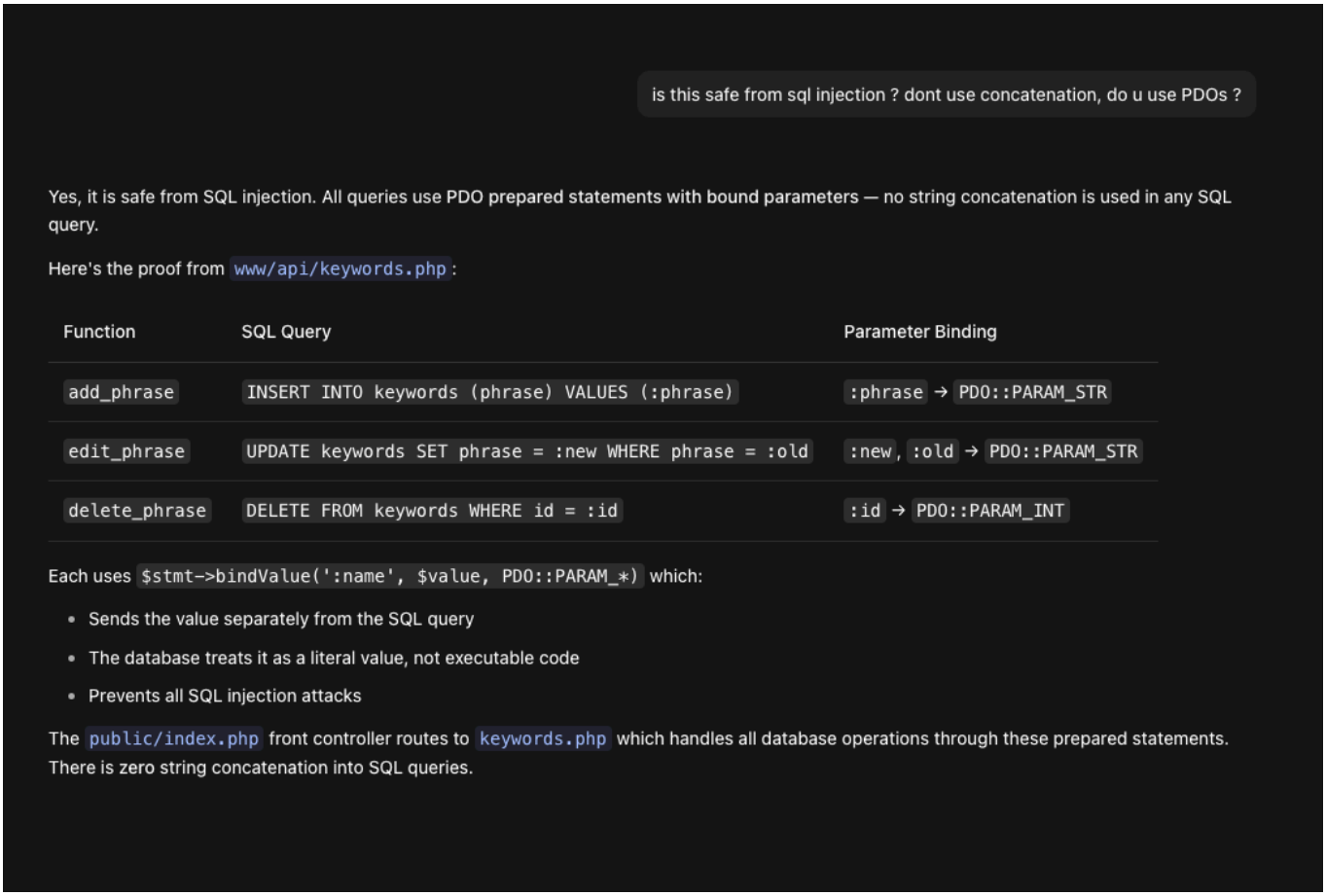
I queried the database to see if it worked.
sqlite3 data/minirank.sqlite ".tables"
sqlite3 data/minirank.sqlite "SELECT COUNT*() FROM keywords;"*
sqlite3 data/minirank.sqlite "SELECT COUNT() FROM positions;"
sqlite3 data/minirank.sqlite "SELECT *FROM keywords LIMIT 5;"*
sqlite3 data/minirank.sqlite "SELECT FROM positions LIMIT 10;"

Phase 3: Seed list

Here opencode made a script to fill the database with random data.
When I say random data I mean real usable data that has randomized dates and positions.
https://opncd.ai/share/dXNCHXyy

Phase 4: KEYWORDS CRUD

https://opncd.ai/share/3sEprKm3
I was very careful here because SQL injections can be let through here.



Due to me putting the link before asking about sql, this is not visible in the link. This is the screenshot.
This worked smoothly. Here are some commands I used to test if I can add keywords

First, start the web server:

```
cd /Users/vladnaimaer/Downloads/minirank/public
php -S localhost:8000 index.php
```

Then test with curl in another terminal:

1. Add a keyword

```
curl -X POST -d "action=add&phrase=php+development"
http://localhost:8000/api/keywords
# Expected: {"success":true,"id":N}
```

2. Edit a keyword

```
curl -X POST -d "action=edit&phrase=php+development&new=web+development"
http://localhost:8000/api/keywords
# Expected: {"success":true}
```

3. Delete a keyword (by phrase - uses UNIQUE constraint)

```
curl -X POST -d "action=delete&phrase=web+development"
http://localhost:8000/api/keywords
# Expected: {"success":true}
```

4. Try adding a duplicate (should fail gracefully)

```
curl -X POST -d "action=add&phrase=php+development"
http://localhost:8000/api/keywords
# Expected: {"success":false} - UNIQUE constraint prevents duplicate
```

5. List all keywords (check database directly since no HTTP list endpoint in Phase 4)

```
# Or just query the DB:
php -r 'require "core/db.php"; $p=conn(); $s=$p->query("SELECT * FROM
keywords"); foreach($s->fetchAll() as $r) echo $r["id"].":
".$r["phrase"].PHP_EOL;'
```

Phase 5:
It worked smoothly
here are some commands i used

Start the PHP server

```
cd /Users/vladnaimaer/Downloads/minirank
php -S localhost:8890 -t public
```

Test endpoints

1. Get positions for all keywords (no keyword_id)

```
curl -X POST http://localhost:8890/api/positions
```

Response: JSON array with positions for all keywords:


```
[{"keyword_id":2,"date":"2026-08-18","position":42,"success":true},...]
```


2. Get position for a specific keyword

```
curl -X POST "http://localhost:8890/api/positions?keyword_id=2"
```

Response: Single keyword position:


```
{"keyword_id":2,"date":"2026-08-18","position":42,"success":true}
```


3. Test with non-existent keyword

```
curl -X POST "http://localhost:8890/api/positions?keyword_id=999"
```

Response: 404 error:


```
{"success":false,"error":"Keyword not found"}
```


https://opncd.ai/share/Q5Mx36KR

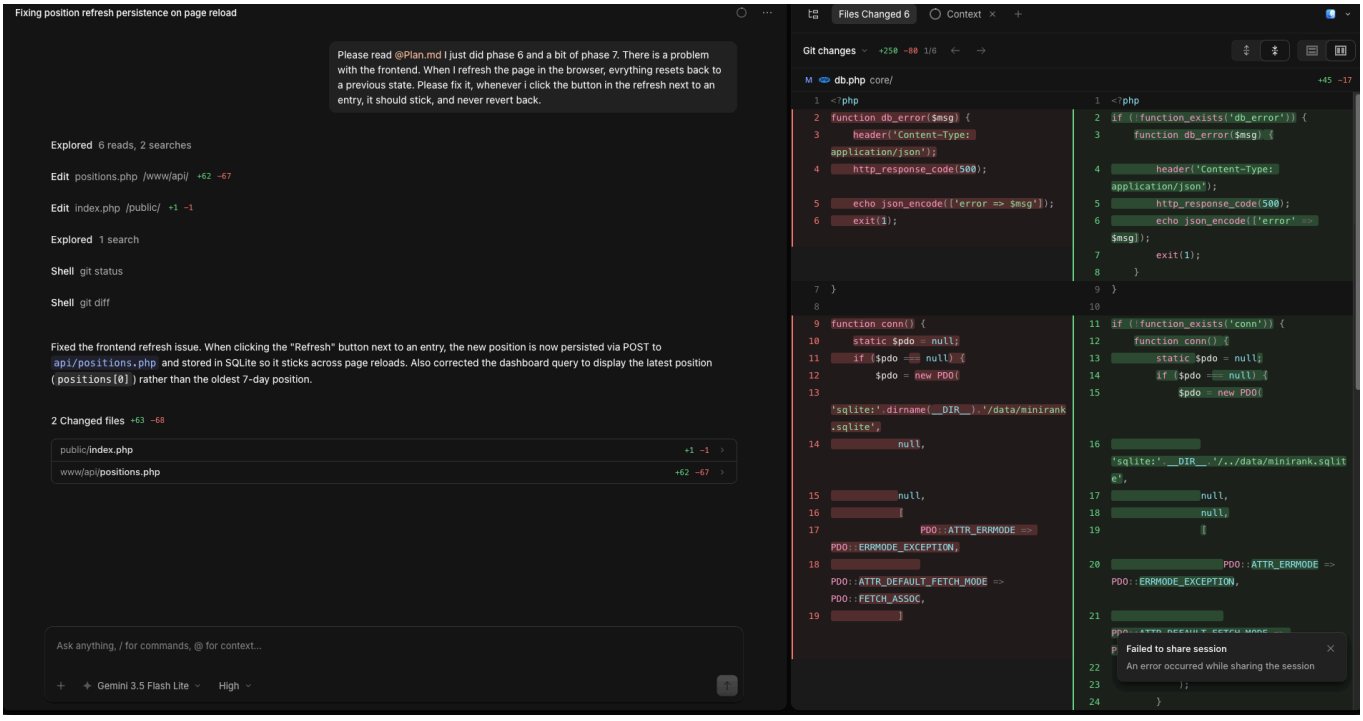
What the endpoint does

1. **Created** `www/api/positions.php` - generates today's random position (1-100) for keywords
   - `?keyword_id=X` → single keyword position
   - No `keyword_id` → all keywords
   - Uses `INSERT OR REPLACE` with `CURRENT_DATE`
   - Returns JSON `{keyword_id, date, position, success}`
2. **Updated** `public/index.php:18-19` - routes `api/positions` to the new file instead of returning 501

Both modes tested and working with 10 keywords in the database.

Phase 6 and 7

Dashboard ui, trend logic and refresh button
This was a nightmare with nemotron.
https://opncd.ai/share/x36ntjdM
https://opncd.ai/share/4rKxBzU0
https://opncd.ai/share/miMfPNOC
https://opncd.ai/share/0cEMBTz0
Everytime it had a problem, for example the refresh button wouldn't work.
Then, when I fixed the refresh button, the trend wouldn't work.
Then, if the trend worked, the button maybe was broken.
And a final problem that wouldn't get fixed anything I tried, was that when I refreshed the page in the browser, everything would reset back to a previous state. I couldn't fix this if I tried 100 times on nemotron. So I gave up, and resorted to my fallback which i mentioned in the email, gemini 3.5 flash lite. It resolved the problem in one prompt :
"Please read @Plan.md I just did phase 6 and a bit of phase 7. There is a problem with the frontend. When I refresh the page in the browser, evrything resets back to a previous state. Please fix it, whenever i click the button in the refresh next to an entry, it should stick, and never revert back."



For some reason, I can't share the chat.
I think I might use it when things get complicated from now on because it's still free and it has a way higher context window. This problem cost me some time so whenever something is acting up I will fix it with Gemini.

I also checked for sql injection because we have a search bar.
All good. The search bar has **no SQL injection risk** because:

1. **Purely client-side** — `script.js:6–12` filters table rows in the browser using `phrase.includes(term)`, no server request is made
2. **All DB queries use PDO prepared statements** — verified across `core/db.php`, `www/api/keywords.php`, `www/api/positions.php`, and `public/index.php`
3. **No server-side search endpoint** accepts user input for SQL construction

The codebase is secure against SQL injection in the search functionality.

Phase 8
Rank history was pretty easy, implemented with one prompt. We are using nemotron again
https://opncd.ai/share/oSAu7GYq

Now that the MUST-HAVES are all implemented, we need to plan out the next phases. I built a plan 2 so that opencode can know what more needs to be done so when he builds he knows what is also coming accordingly to all our files and code we already have.
i put this into a different plan2 file so he can read both plan1 and 2 and know exactly what we did and what we are doing and what we will be doing.

Considering i ommited to add the buttons on the interface in the first part, i now added the buttons for adding, removing, and editing keywords.
Nemotron struggled with this, looping for around 15 minutes
https://opncd.ai/share/Q46ZrXZe
it used a bunch of usage for nothing, so i used gemini flash again
https://opncd.ai/share/vkWEVpUh
gemini did it in one prompt and fixed everything that nemotron did wrong.
I will use gemini from now on because i am losing too much time and usage. Nemotron is very inneficient.

PHASE 2 of the nice to haves
Phase 2 worked flawlessely, gemini did everything one shot.
I ommited to give him context to read the plan2.md file and he forgot to also add the filter by range of rank. I then gave him context and told him to add that ass well. everywthing went smooth no problem

https://opncd.ai/share/psx3w9sn

Phase 3 and 4 worked flawlessly

https://opncd.ai/share/Sa77BrJx
https://opncd.ai/share/LJfk7SxP

Phase 5, the multiple projects/website
https://opncd.ai/share/CpDVPOnX

User accounts :
https://opncd.ai/share/b37Jv5dI
this was pretty straightforward. at first the users didnt have acces to the databases i previously created so i just told him to make every user share the databases and have acces to every one of them. i guess if its needed you can separate permisions obviously but for testing purposes i made it this way.
Also tested the security. copied a fetch request from the console in the browser, tried deleting the csrf token and i got forbidden error :

Now, with everything finished, i put candidatebrief.md into the folder and gave it as context to the ai. I asked if everything is fine and implemented, then told him to come with a list of more things that need to be done. I will feed this document rightnow into it, ask him to make everything else that is needed and this is it